SWE 4101

Object Oriented Concepts I

Prepared By:
Samia Nawsheen
Junior Lecturer, CSE

Table of contents

Code Snippet

```
public class Demo {
    public static void main(String[] args) {
        int[] marks = {80, 90, 75};
        System.out.println("Marks fetched successfully");
        System.out.println(marks[5]); // Boom
        System.out.println("Report generated"); // Never runs
    }
}
```

What do you think happens after the line with marks[5]?

"Report generated" never printed. One bad line took down the whole program.

What is an Exception?

- An **exception** is an event that disrupts the normal flow of a program during execution

- It's Java's way of saying: *"Something unexpected happened, and I don't know what to do next, you decide."*

- **Analogy:** You're driving using GPS directions (normal flow). Suddenly there's a pothole (exception). You don't crash the car. You steer around it and continue the trip. The steering = exception handling

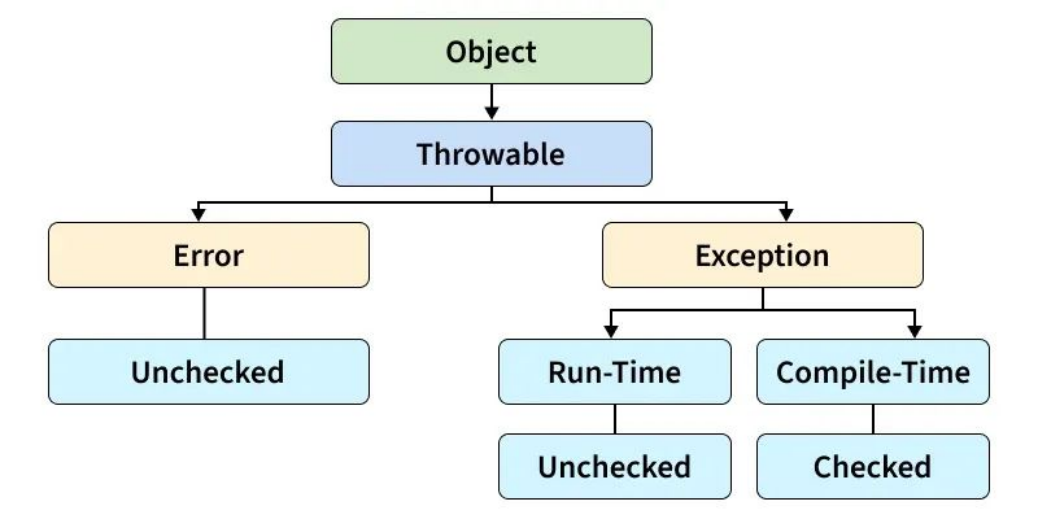
Why handle exceptions?

Without handling: **one mistake kills the entire program**, even unrelated code after it

With handling, program can

- Recover and continue
- Show an informative message
- Clean up resources (files, DB connections) before exiting
- Log what went wrong for developers to debug later

Java Exception Hierarchy



Error: Represents a fatal system-level crisis that an application cannot recover from

Exception: Represents an application-level problem that a programmer can anticipate and handle

Errors vs Exceptions

Feature	Error	Exception
Recoverability	**Irrecoverable**. The program must terminate.	**Recoverable**. Can be handled to keep running.
Root Cause	Environment, system resources, or the JVM.	Application logic, inputs, or external APIs.
Type	Always **Unchecked** (Compile-time blind).	Can be **Checked** or **Unchecked**.
Handling Rule	Do **not** try to catch them.	Expected to be handled via try-catch.
Examples	OutOfMemoryError, StackOverflowError	NullPointerException, IOException

Checked vs Unchecked Exceptions

Feature	Checked Exception	Unchecked Exception
When is it checked?	Compile time (before running).	Runtime (while running).
Are you forced to handle it?	**Yes**. Code will not compile without handling it.	**No**. Handling is optional.
Typical Causes	Situations that your program cannot fully control (missing files, network failures, database errors, user input/output problems).	Bugs in the program, such as invalid logic or improper use of objects.
Java Class Parent	Extends Exception.	Extends RuntimeException.
Examples	IOException, FileNotFoundException.	NullPointerException, ArithmeticException.

try-catch

```
try {
    // risky code that might throw an exception
} catch (ExceptionType e) {
    // code that runs if that exception occurs
}
```

- try block: the code you're "watching"
- catch block: the "plan B", only runs if a matching exception is thrown
- If nothing goes wrong in try, the catch block is simply skipped

try-catch in action

```
public class Divide {
    public static void main(String[] args) {
        try {
            int result = 10 / 0;
            System.out.println(result);
        } catch (ArithmeticException e) {
            System.out.println("Can't divide by zero!");
        }
        System.out.println("Program continues normally...");
    }
}
```

The line after the risky code still runs

Multiple catch blocks

```
try {
    int[] arr = new int[3];
    arr[5] = 10;          // ArrayIndexOutOfBoundsException
    int x = 10 / 0;       // never reached
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Array index problem: " + e.getMessage());
} catch (ArithmeticException e) {
    System.out.println("Math problem: " + e.getMessage());
} catch (Exception e) {
    System.out.println("Some other problem occurred");
}
```

Java checks them top to bottom and runs the first match

Ordering Rule

- More specific exceptions must come *before* more general ones

```
catch (Exception e) { ... }
catch (ArithmeticException e) { ... } // Compile error
```

- Exception is the parent of ArithmeticException, NullPointerException, etc.
- The general Exception catch already matches everything, so the specific one below can never be reached — Java flags it as an error, not just a warning

Multiple catch shortcut

```
try {
    // risky code
} catch (ArithmeticException | NullPointerException e) {
    System.out.println("Either a math or null error: " +
e.getMessage());
}
```

- This is allowed only from Java 7 onwards.
- Use the pipe symbol | to handle multiple exception types the same way, without repeating code
- Only valid when you want to handle them identically

No Inheritance Rule

- When using the single-line multi-catch syntax, you must not list exceptions that share a parent-child relationship
- All exception types listed within a single pipe-delimited statement must be completely independent alternatives

```
// WRONG: Compilation Error!
// ArithmeticException is a subclass of RuntimeException
catch (ArithmeticException | RuntimeException e) {
    // ...

}
```

finally Block

```
try {
    System.out.println("In try");
    int x = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("In catch");
} finally {
    System.out.println("In finally - always runs");
}
```

- finally runs no matter what — whether an exception occurred, was caught, or not
- Even if there's a return statement inside try or catch, finally still executes before the method actually returns
- Common use: closing files, database connections, releasing resources

Common Built-in Exceptions

Exception	When it happens
NullPointerException	Calling a method or accessing a field on a null reference
ArrayIndexOutOfBoundsException	Accessing an array index that is outside the valid range
ArithmeticException	Performing an illegal arithmetic operation (e.g., dividing an int by zero)
NumberFormatException	Converting an invalid string to a number (e.g., Integer.parseInt("abc"))
ClassCastException	Performing an invalid type cast between incompatible object types
IllegalArgumentException	Passing an argument to a method that it does not accept
IOException *(checked)*	File or input/output operations fail (e.g., file not found, read/write error)

throw: Triggering an Exception Yourself

```
public class Age {
    static void checkAge(int age) {
        if (age < 18) {
            throw new IllegalArgumentException("Underage");
        }
        System.out.println("Access granted");
    }
}
```

- **‘throw’** is used inside a method to manually create and raise an exception
- Followed by exactly one exception object (**new SomeException(...)**)
- Once thrown, normal execution stops immediately and Java looks for a matching catch

throws: Declaring a Risk

```
public void readFile(String path) throws IOException {
    FileReader file = new FileReader(path);
    // ...
}
```

- ‘**throws**’ goes in the method signature and acts as a warning label, not an action
- It tells callers: "**Heads up! calling this method might cause this checked exception. Handle it or pass it on.**"
- Required for checked exceptions if you're not catching them inside the method

throw vs throws

Feature	throw	throws
Purpose	Explicitly throws an exception object.	Declares the exceptions a method may throw.
Location	Inside the method body.	In the method signature (after the parameter list).
Followed by	An exception object (instance of an exception class).	One or more exception class names.
Quantity	Can throw only one exception object in a single throw statement.	Can declare multiple exception types, separated by commas.
Execution	Immediately transfers control to the nearest matching catch block (or terminates the method if uncaught).	Does **not** affect execution; it only informs the compiler and callers that the method may throw these exceptions.

Exception Propagation

```
public static void methodA() throws IOException {
    methodB();
}

public static void methodB() throws IOException {
    throw new IOException("Error");
}

public static void main(String[] args) {
    try {
        methodA();
    } catch (IOException e) {
        System.out.println("Handled in main.");
    }
}
```

try-with-resources

Here's a typical try-catch-finally block:

```
FileReader reader = null;
try {
    reader = new FileReader("test.txt");
    // do something
} catch (FileNotFoundException e) {
    System.out.println("Error!");
} finally {
    try {
        if (reader != null) {
            reader.close();
        }
    } catch (IOException e) {
        System.out.println("Couldn't close the file.");
    }
}
```

- Easy to forget to close a resource this way
- Lots of nested try-catch

try-with-resources

```
try (FileReader reader = new FileReader("test.txt")) {
    // do something
} catch (IOException e) {
    System.out.println("Error!");
}
```

- Declare the resource inside try (...)
- Java automatically calls .close() when the block ends, regardless of success or failure

Custom Exceptions

- Built-in exceptions are generic — IllegalArgumentException doesn't tell you which business rule was broken
- Custom exceptions give meaningful, domain-specific names:
  - InsufficientBalanceException
  - InvalidStudentIdException
  - SeatNotAvailableException
- Makes error handling and debugging far more readable than a bunch of generic exceptions

Custom Checked Exception

```
public class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

public class Account {
    double balance = 500;

    void withdraw(double amount) throws InsufficientBalanceException {
        if (amount > balance) {
            throw new InsufficientBalanceException("Insufficient balance!");
        }
        balance -= amount;
    }
}
```

- Extend Exception → makes it **checked** → caller is forced to handle or declare it
- Use when the caller genuinely **should be forced** to deal with the situation (e.g., a banking rule)

Custom Unchecked Exception

```
public class InvalidStudentIdException extends RuntimeException {
    public InvalidStudentIdException(String message) {
        super(message);
    }
}
```

- Extend RuntimeException → makes it unchecked → caller isn't forced to catch it
- Use when the error is just a programming bug the caller should fix
- Ask yourself: "Would I want every caller to be forced to write a try-catch for this?"
    - Yes → checked.
    - No, it's more of a bug → unchecked.

Pop Quiz

```
int[] arr = {1,2,3};
System.out.println(arr[5]);
```

ArrayIndexOutOfBoundsException

```
String s = null;
System.out.println(s.length());
```

NullPointerException

```
int x = 10 / 0;
```

ArithmeticException

Pop Quiz

```
FileReader f = new
FileReader("abc.txt");
```

FileNotFoundException

```
Integer.parseInt("abc");
```

NumberFormatException

```
Animal animal = new Animal();
Dog dog = (Dog) animal;
```

ClassCastException

Pop Quiz

```
try {
    System.out.println("A");
    int x = 10 / 0;
    System.out.println("B");
} catch (ArithmeticException e) {
    System.out.println("C");
} finally {
    System.out.println("D");
}
System.out.println("E");
```

A
C
D
E

1. System.out.println("A") → prints **A**
2. 10 / 0 → ArithmeticException is thrown
3. System.out.println("B") → **skipped completely**
4. Java jumps to the matching catch → prints **C**
5. finally always executes → prints **D**
6. The exception has been handled, so execution continues after the try-catch-finally block → prints **E**

Thank You